

D0000R0: Extensible Scoped Enumerations

Author: Andrey Fokin
Email: lazzyfox@gmail.com
Date: 2026-04-20
Project: Programming Language C++
Audience: Core Working Group
Abstract

This paper proposes an extension to scoped enumerations that allows one enumeration to reuse the enumerators of one or more existing scoped enumerations and add new enumerators while preserving strong typing and avoiding duplication. Idea was being discussed in - Add inheritance for Enum Class enumerations topic (<https://lists.isocpp.org/std-proposals/2026/04/18118.php>)

1. Introduction

There are practical cases where several enumeration types share a common base set of values and differ only by a few additional values. In such cases, the same enumerator list is repeated across multiple definitions, which increases maintenance cost and makes the relationship between variants less explicit.

For example, a traffic light controller may have a common set of states {Red, Yellow, Green} and several variants that add additional signals such as {LeftArrow} or {RightArrow}. Today, each variant must be defined separately, even when most of the enumerators are identical.

2. Motivation and Scope

The goal of this proposal is to reduce duplication and make related enumerations easier to maintain. A shared base enumeration could describe the common states, while extended enumerations could add additional states specific to a variant.

This proposal does not attempt to introduce object-oriented runtime polymorphism for enumerations. It also does not change the behavior of unscoped enumerations. The intent is limited to a controlled form of extension for scoped enumerations.

3. Problem Statement

Consider the following repeated definitions:

```
cpp
enum class TrafficLight : std::uint8_t { Red, Yellow, Green };

enum class TrafficLightLeftArrow : std::uint8_t { Red, Yellow, Green,
LeftArrow };

enum class TrafficLightRightArrow : std::uint8_t { Red, Yellow, Green,
RightArrow };

enum class TrafficLightTwoArrows : std::uint8_t { Red, Yellow, Green, LeftArrow,
RightArrow };
```

These declarations duplicate the same base enumerators in every variant. If the shared set changes, every related type must be updated manually. This creates avoidable repetition and increases the chance of inconsistencies.

4. Proposed Solution

This paper proposes a language facility that allows a scoped enumeration to declare one or more base scoped enumerations and extend them with additional enumerators.

A possible syntax could be:

```
cpp
enum class TrafficLight : std::uint8_t { Red, Yellow, Green };

enum class TrafficLightLeftArrow : TrafficLight { LeftArrow };

enum class TrafficLightRightArrow : TrafficLight { RightArrow };

enum class TrafficLightTwoArrows : TrafficLight { LeftArrow, RightArrow };
```

In this model, the extended enumeration reuses the enumerators of the base enumeration and adds its own enumerators. The exact rules for type identity, underlying type, name lookup, conversions, and storage representation would need to be specified precisely.

5. Multiple Base Enumerations

A derived scoped enumeration may also reuse enumerators from more than one base scoped enumeration.

For example:

```
cpp
enum class BaseColor : std::uint8_t { Red, Yellow, Green };
enum class ArrowSignal : std::uint8_t { LeftArrow, RightArrow };

enum class TrafficLightWithArrows : BaseColor, ArrowSignal {
    LeftArrow,
    RightArrow
};
```

This form may be useful when a new enumeration is composed from two independent enumerator sets. However, it raises additional design questions, including name conflicts, value uniqueness, and underlying type compatibility.

6. Error Handling and Invalid Forms

Any invalid declaration under this proposal shall be ill-formed and diagnosed by the implementation.

A program is ill-formed if:

- A derived scoped enumeration reuses an enumerator name that already appears in one of its base enumerations.

- Two base enumerations provide enumerators with the same name, unless the design specifies an explicit disambiguation rule.

- The underlying types of the base enumerations are incompatible, if a common underlying type is required by the final wording.

- A derived enumeration attempts to inherit from a non-scoped enumeration.

- A derived enumeration attempts to inherit from an enumeration that is not eligible as a base under the proposed rules.

- A derived enumeration introduces an enumerator that conflicts with any inherited enumerator.

- A derived enumeration uses more than one base enumeration when the implementation or final wording does not support multiple inheritance of enumerator sets.

If the final wording permits multiple base enumerations, then the following

additional conditions should be specified and enforced:

All base enumerations must have compatible underlying types, or the derived enumeration must declare an explicit underlying type compatible with all bases.

Enumerator names inherited from different bases must be distinct.

Enumerator values inherited from different bases must not conflict, unless the proposal defines a deterministic resolution rule.

Because enumerations are part of the type system, these errors should be diagnosed at compile time rather than deferred to runtime.

7. Design Considerations

Several semantic questions must be resolved before such a feature could be standardized:

Whether the extended enumeration inherits the underlying type of all base enumerations or must declare one explicitly.

Whether enumerators from different bases may share the same name.

Whether enumerator values must remain unique across all base and derived enumerations.

Whether explicit conversions between base and extended enumerations are permitted.

Whether multiple-base extension should be part of the initial design or treated as a later extension.

Whether this feature should be viewed as true inheritance or as enumeration composition.

Because enum class is not currently a class type in the object-oriented sense, the term "inheritance" may be misleading. A more precise description may be "extensible scoped enumerations" or "enumeration composition".

8. Alternatives Considered

A common alternative is to define each enumeration independently and factor out shared names with using enum where the names are needed. This avoids any change to the core language and already supports local name reuse.

Another alternative is to use a class-based representation when more structure is required. That approach, however, changes the data model and may be heavier than needed for simple enumerated states.

9. Impact on the Standard

Standardizing this feature would require changes to the wording for enumeration declarations, name lookup, compatibility, and possibly conversion rules. The interaction with existing scoped enumerations would also need to be defined carefully, since current C++ does not treat enumerations as extensible base types.

10. Conclusion

This proposal aims to reduce repetition and improve clarity when multiple related scoped enumerations share a common set of enumerators. The feature becomes significantly more complex when multiple base enumerations are allowed, so the committee may wish to evaluate the single-base form first and consider multiple-base extension separately.

Notes

This draft is now in a much better WG21 shape, but it still needs a real Proposed Wording section before it can be considered complete for submission. It also may help to add a short Prior Art or Existing Solutions section centered on using enum and current scoped-enum behavior.